

FUNNY LAND

작성자 : 안승현

목차

1. FunnyLand 포트폴리오 소개
2. Photon API
3. 싱글톤 매니저 클래스
4. 씰 클래스
5. 게임 & 컨텐츠 클래스
6. 유닛 클래스
7. 카메라 클래스
8. 무기 클래스

9. 유저 데이터 관리
10. 네트워크 오브젝트 관리
11. 방 목록
12. 아이템
13. 아이템 구매
14. 최후의 생존자 (미니 게임)
15. 좀비 서바이벌 (미니 게임)
16. 플레이 영상 및 깃허브 링크

2. FUNNYLAND 포트폴리오 소개



- 사용 엔진 : UNITY
- 엔진 버전 : 2020.3.19f1 LTS
- 사용 언어 : C#
- 작업 인원 : 1명
- 작업 영역 : 콘텐츠 제작, 디자인, 기획
- 장르 : 미니 게임
- 소개 : Photon Api + 유니티로 만든 멀티플레이 미니게임
- 플랫폼 : PC
- 형상 관리 : GitHub



3. PHOTON API

- 네트워크 API
- 클라우드 기반, Listen Server 구조
- 서버 인원은 무료는 10~20명 지원, 유료는 100이상 지원
- 서비스 종류는 pun, bolt, quantum 등 이중 PUN을 이용해 포트폴리오 제작

✓ Photon API를 효율적으로 사용하기 위해서
Photon API 문서, Discord, 유튜브 참고하여 개념 학습 (Replication, RPC, PhotonView)

PHTON PUN 구조

Client
Client
Client
Client



Name Server

APP ID, Game Ver, 지역



Master Server

Looby, 방 생성 입장



Game Server

멀티 플레이 컨텐츠

```
static Managers instacne;
참조 13개
public static Managers Instance
{
    get
    {
        Init();
        return instacne;
    }
}
```

```
DataManager data = new DataManager();
InputManager input = new InputManager();
PhotonNetworkManager photon = new PhotonNetworkManager();
PoolManager pool = new PoolManager();
ResourceManager resource = new ResourceManager();
SceneManagerEx scene = new SceneManagerEx();
SoundManager sound = new SoundManager();
ParticleManager particle = new ParticleManager();
UIManager ui = new UIManager();
```

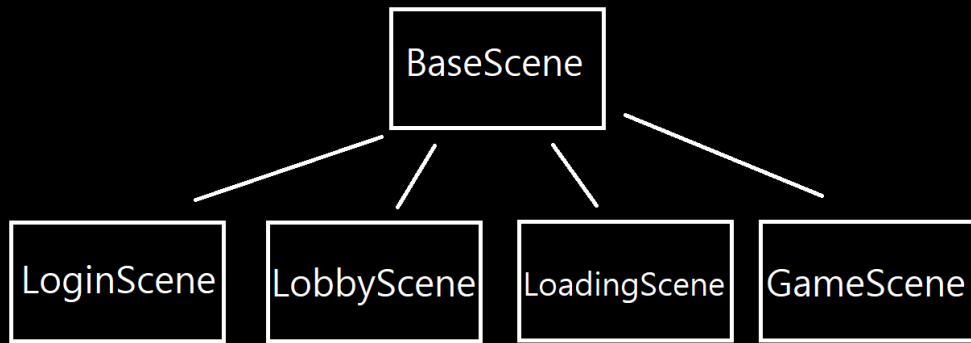
```
참조 90개
public static DataManager Data { get { return Instance.data; } }
참조 58개
public static InputManager Input { get { return Instance.input; } }
참조 26개
public static PhotonNetworkManager Photon { get { return Instance.photon; } }
참조 8개
public static PoolManager Pool { get { return Instance.pool; } }
참조 55개
public static ResourceManager Resource { get { return Instance.resource; } }
참조 23개
public static SceneManagerEx Scene { get { return Instance.scene; } }
참조 99+개
public static SoundManager Sound { get { return Instance.sound; } }
참조 1개
public static ParticleManager Paticle { get { return Instance.particle; } }
참조 80개
public static UIManager UI { get { return Instance.ui; } }
```

3. 싱글톤 매니저 클래스

- 유니티 엔진은 다용도 엔진이라 언리얼 엔진에 비해 제공하는 게임 콘텐츠 기능이 적기 때문이다.
- 모든 매니저 클래스를 관리 및 담당하는 클래스와 그 특정 기능을 수행하는 전역 클래스 생성
- 클라이언트와 서버가 서로 통신하기 위한 최초의 클래스

소스 코드 : [Managers.cs](#)

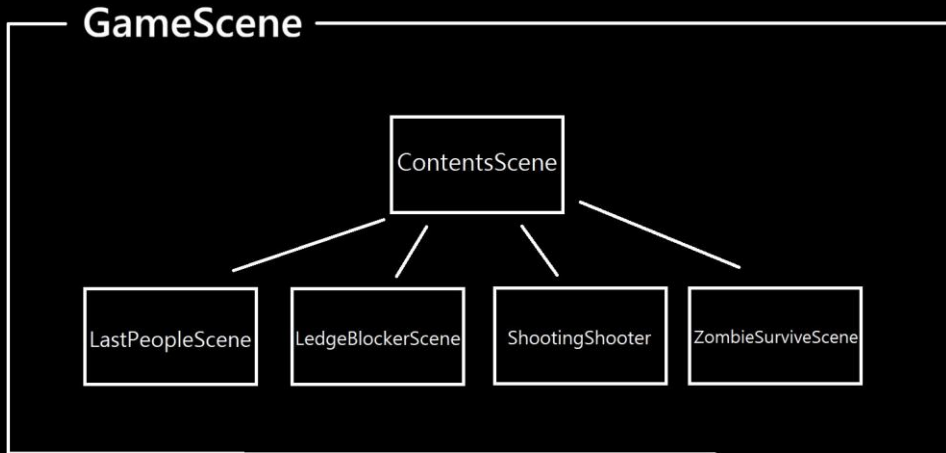
4. 씬 클래스



소스 코드 : [BaseScene.cs](#)

- 씬 들의 공통된 기능들을 묶어 상위클래스로 관리 하기 위함 또는 상세 기능은 하위 클래스에 구현 하여 기능의 다형성을 제공하기 위해서
- 추상 클래스를 선언해 공통적인 기능들은 가상함수로 선언하고 구체적인 구현은 하의 클래스에서 구현
- 로그인, 로비, 씬 로딩, 게임 씬 총 4개의 씬 존재

5. 게임 & 콘텐츠 씬



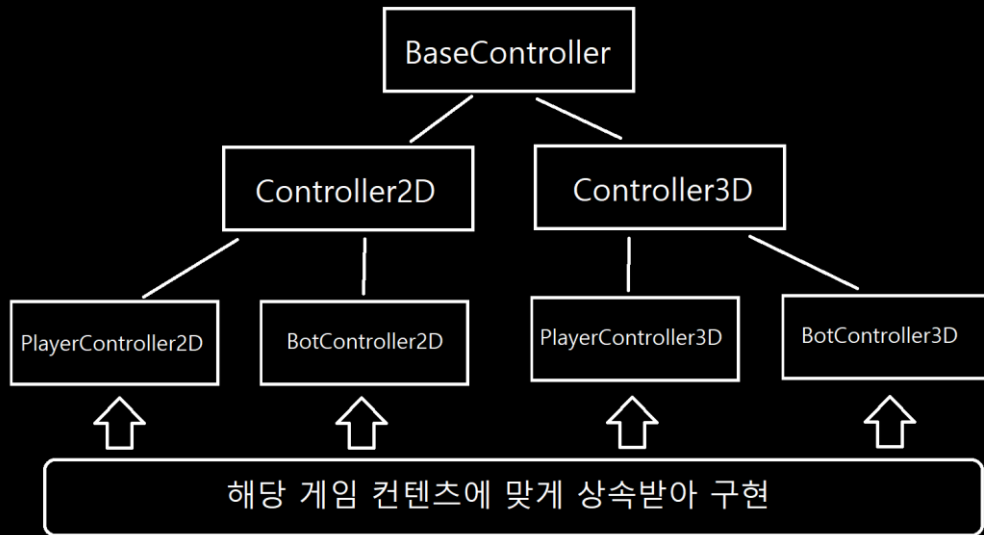
✓ ContentScene은 GameScene에서 동적으로 생성된다.

소스 코드 : [GameScene.cs](#)

소스 코드 : [ContentScene.cs](#)

- GameScene
 - 여러 콘텐츠 씬에서 사용되는 카메라, 조명, UI 를 관리하기 위한 클래스
 - ContentScene을 로드/비로드 및 관리
 - 해당 씬이 활성화시 다른 코드에서도 접근하기 쉽게 전역 변수인 SceneManagerEX에 자기자신을 등록한다.
- ContentScene
 - 여러 미니 게임에서 사용되는 공통된 기능을 묶고 기능들을 추상화해서 관리 및 기능을 다양하게 구현하기 위해
 - 상위 클래스에는 공통된 기능 가상 함수로 구현 자식 클래스에서 기능 확장 및 가상 함수 오버라이드

6. 유닛 클래스



[BaseController.cs](#)

[Controller2D.cs](#)

[Controller3D.cs](#)

[PlayerController2D.cs](#)

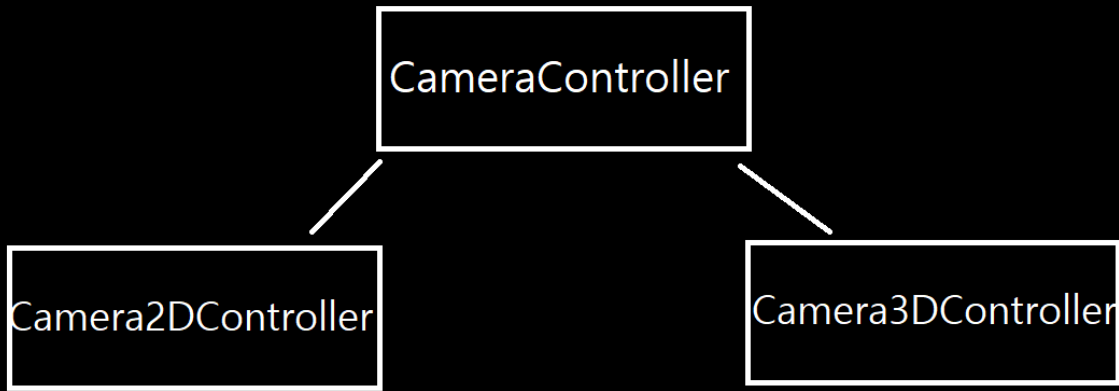
[PlayerController3D.cs](#)

[BotController2D.cs](#)

[BotController3D.cs](#)

- 여러 미니게임을 구현하기 위해서는 매번 미니 게임에 맞게 클래스를 생성 해야 하는데 매번 모든 기능을 구현하기에는 너무 많은 작업이 필요하다 그래서 자주 사용되고 공통적인 기능들을 묶어 계층별로 클래스 생성
- 오브젝트를 조작과 데이터를 종합적으로 관리하는 BaseController 추상 클래스 기점으로 각 기능과 목적에 맞게 클래스 분리 구현 콘텐츠 기호에 맞게 하위 클래스를 상속 받아 구현하면 된다.

7. 카메라 클래스



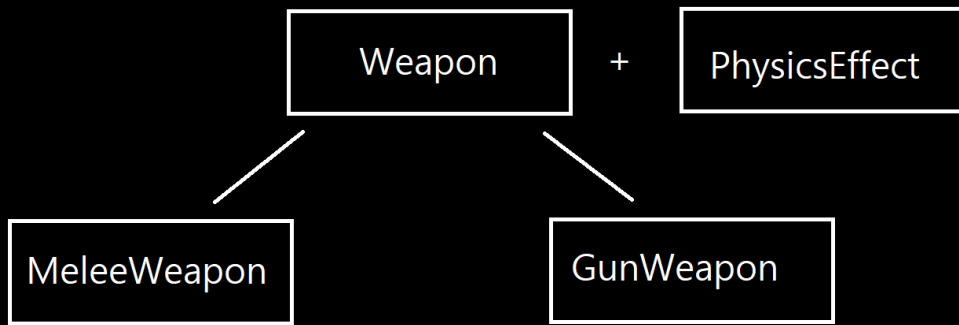
소스 코드 : [CameraController.cs](#)

[Camera2DController.cs](#)

[Camera3DController.cs](#)

- 2D, 3D 카메라 시점과 제어 방법이 다르기 때문에 계층별 클래스 생성
- 전반적인 카메라 설정하는 CameraController 추상 클래스,
- 해당 추상 클래스를 상속받아 2D 혹은 3D 콘텐츠에서 해당 수치만 변경하여 사용할 수 있도록 구현

8. 무기 클래스



소스 코드 : [Weapon.cs](#)

[MeleeWeapon.cs](#) [GunWeapon.cs](#)

- 무기를 사용하는 특정 콘텐츠를 위해 범용적으로 사용 할 수 있도록 하기 위해서
- 무기 클래스와 그리고 자식 클래스인 근접무기, 원거리 무기 클래스 생성

```

T LoadJsonBackEnd<T>(T original, string tableName, string columnName)
{
    var bro = Backend.GameData.GetMyData(tableName, new Where());
    if(bro.IsSuccess())
    {
        string dataText;
        if (bro.GetReturnValuetetoJSON()["rows"].Count > 0)
        {
            dataText = bro.FlattenRows()[0][columnName].ToString();
        }
        else // 없으면 초기화
        {
            dataText = JsonUtility.ToJson(original);

            Param param = new Param();
            param.Add(columnName, dataText);

            Backend.GameData.Insert(tableName, param);
        }

        return JsonUtility.FromJson<T>(dataText);
    }
}

```

■ 데이터 불러오기

```

public bool SaveBackEndData<T>(T dataClass, string tableName, string columnName)
    where T : class
{
    string dataText = JsonUtility.ToJson(dataClass);

    Param param = new Param();
    param.Add(columnName, dataText);

    Where where = new Where();
    where.Equal("owner_inDate", Backend.UserInDate);

    var result = Backend.GameData.Update(tableName, where, param);
    return result.IsSuccess();
}

```

■ 데이터 저장하기

9. 유저 데이터 관리

- 네트워크 게임의 유저 데이터를 저장하기 위해
- 유저 데이터를 저장/로드 및 관리하기 위한 BackEnd API 사용
- 데이터
 - 데이터 불러오기
 - 대상 테이블 데이터를 JSON으로 받아 역직렬화 후 객체에 삽입
 - 데이터 저장
 - 대상 테이블 데이터에 객체 데이터를 JSON으로 변환 하여 전송

소스 코드 : [DataManager.cs](#)

10. 네트워크 오브젝트 관리

```
public void SetPhotonObject(GameObject go)
{
    if(go.TryGetComponent(out PhotonViewEx PV)
    {
        if(PV.IsRoomView) // 룸 오브젝트
            SetRoomObj(go);
        else // 플레이어 오브젝트
            SetPlayer(go);
    }
}
```

참조 3개

```
public GameObject GetPhotonObject(int viewID)
{
    if (viewID < 1000) // 룸 오브젝트
        return GetRoomObj(viewID);
    else // 플레이어 오브젝트
        return GetPlayerObject(viewID);
}
```

참조 3개

```
public string GetObjectNickName(int viewID)
{
    if (viewID < 1000) // 룸 오브젝트
        return GetRoomObj(viewID).name;
    else // 플레이어 오브젝트
        return GetPlayer(viewID).NickName;
}
```

- 현재 씬 에서 네트워크 오브젝트를 쉽게 찾기 위한 게임 매니저 클래스에서 함수 기능 구현
- 게임 매니저는 싱글톤 매니저의 전역변수로 선언되어 손쉽게 접근 가능
- 현재 씬의 유저 오브젝트, 룸 오브젝트를 관리
- 해당 오브젝트들을 Creature클래스에서 상속받은 클래스가 활성화 될 때 ViewID 기반으로 오브젝트들이 직접 등록한다.

소스 코드 : [GameManager.cs](#)

11. 방목록

```
public override void OnRoomListUpdate(List<RoomInfo> roomList)
{
    int roomCount = roomList.Count;
    for (int i = 0; i < roomCount; i++)
    {
        // 해당 방은 제거되지 않았다.
        if (!roomList[i].RemovedFromList)
        {
            // 새로운 방 추가
            if (!myRoomList.Contains(roomList[i]))
                myRoomList.Add(roomList[i]);
            // 방이 수정된 경우 해당 인덱스에 변경된 방 정보 덮어쓰기
            else
                myRoomList[myRoomList.IndexOf(roomList[i])] = roomList[i];
        }
        // 해당 방을 리스트에서 제거
        else if (myRoomList.IndexOf(roomList[i]) != -1)
            myRoomList.RemoveAt(myRoomList.IndexOf(roomList[i]));
    }

    if (findRoom != null)
        findRoom.RoomInit(myRoomList);
    if (roomPW != null)
        roomPW.RoomPWInit(myRoomList.Find(room => room.Name == roomPW.roomInfo.Name));
}
```

- 생성된 방을 로비에 있는 유저가 보거나 혹은 직접 참여하기 위함
- 포톤에서 제공하는 OnRoomListUpdate 함수를 통해 방 데이터가 변경된 경우 콜백함수로 호출된다.
- 해당 방목록 데이터를 정리한다음 해당 UI에게 데이터를 넘겨 방목록을 표시하도록 구현

소스 코드 : [UI_LobbyScene.cs](#)

12. 아이템

[Serializable]

참조 31개

```
public abstract class ItemData
```

```
{
```

```
    public int Id;
```

```
    public string Name;
```

```
    public string Image;
```

```
    public string Tooltip;
```

```
    public int Price;
```

```
    public ItemRating Rating;
```

```
    public GoodsType Goods;
```

```
    public ItemType _ItemType;
```

참조 7개

```
    public abstract Item CreateItem();
```

```
}
```

```
"icons": [
  {
    "Id": 3000,
    "Name": "남성 이미지",
    "Image": "default",
    "Tooltip": "기본 제공하는 이미지 입니다.",
    "Price": 0,
    "Goods":1,
    "Rating":0
  },
  {
    "Id": 3001,
    "Name": "여성 이미지",
    "Image": "icon1",
    "Tooltip": "기본 제공하는 이미지 입니다.",
    "Price": 0,
    "Goods":1,
    "Rating":0
  },
]
```

- 플레이어 개성을 표현하기 위해 구현
- 아이템 데이터 추상 클래스 정의
- ItemData 추상 클래스를 상속받아 칭호, 아이콘, 기타 아이템 클래스 생성
- 게임 시작 시 JSON의 데이터를 역직렬화 해서 메모리에 적재

소스 코드 : [Data.Contents.cs](#) 156~

13. 아이템 구매

```
public void ShowPurchaseWindow(int id, Text priceText)
{
    if (Managers.Data.ItemDict.TryGetValue(id, out Data.ItemData _ItemData))
    {
        UI_Notice notice = Util.SimplePopup($"({_ItemData.Name})을 구매하시겠습니까. Wn 구매비용은" +
            $" {_ItemData.Price} {Util.GetGoodsTypeToString(_ItemData.Goods)} 입니다.");
        notice.Get<Button>((int)UI_Notice.Buttons.OKButton).gameObject.CoverBindEvent((data) =>
        {
            Managers.Sound.Play2D("SFX/UI_Click", Define.Sound2D.Effect2D);

            if(Util.PurchaseItemResult(_ItemData.Goods, _ItemData.Price))
            {
                gameObject.RemoveAllEvent();
                notice.ClosePopupUI();
                Managers.Data.PlayerInventoryData.itemId.Add(_ItemData.Id);
                Managers.Data.SaveBackEndData(Managers.Data.PlayerInventoryData,
                    "PlayerInventory", "playerInventoryToJson");
                Managers.Sound.Play2D("SFX/UI_PurchaseCompleted");
                Util.SimplePopup("구매이 완료되었습니다.");
                priceText.text = $"보유중";
            }
            else
            {
                Managers.Sound.Play2D("SFX/UI_AlertSound");
                Util.SimplePopup("비용이 부족합니다.");
            }
        });
        notice.ActiveCancelBtn();
    }
}
```

소스 코드 : [UI_Store.cs](#)

```
public static bool PurchaseItemResult(Define.GoodsType goosType, int itemPrice)
{
    switch(goosType)
    {
        case Define.GoodsType.COIN:
            if (itemPrice <= Managers.Data.PlayerInfoData.coin)
            {
                Managers.Data.PlayerInfoData.coin -= itemPrice;
                Managers.Data.SaveBackEndData(Managers.Data.PlayerInfoData, "PlayerInfo", "playerInfoToJson");
                Managers.UI.SceneUI.UpdateSyncUI?.Invoke();
                return true;
            }
            break;
        case Define.GoodsType.DIA:
            if (itemPrice <= Managers.Data.PlayerInfoData.dia)
            {
                Managers.Data.PlayerInfoData.dia -= itemPrice;
                Managers.Data.SaveBackEndData(Managers.Data.PlayerInfoData, "PlayerInfo", "playerInfoToJson");
                Managers.UI.SceneUI.UpdateSyncUI?.Invoke();
                return true;
            }
            break;
    }

    return false;
}
```

소스 코드 : [Util.cs](#)

- 구현된 아이템을 비용을 지불하여 플레이어가 소유하게 하기 위해 구현
- Coin은 게임을 통해서만 얻을 수 있으며 Coin을 사용하여 아이템을 구매 할 수 있습니다.
- Coin 수치 변경 + 구매한 아이템 ID가 추가된 후 BackEnd에 반영됩니다.

14. 최후의 생존자 (미니 게임)



- 소개
 - 플레이어와 AI 사이에서 플레이어를 찾아 제거하면 되는 심플한 게임입니다.
- 특징
 - 자기장이 존재하며 특정 시간마다 줄어 듭니다. (최대 5번)
 - 자기장에 닿으면 화면이 파랗게 변하고 체력이 서서히 닳습니다.
 - 레드존 존재 매 60초마다 랜덤으로 발생합니다.
 - 플레이어가 아닌 AI를 제거할 시 자신의 체력이 닳습니다.
- 승리 조건
 - 상대 플레이어를 모두 제거

자기장 기능 구현

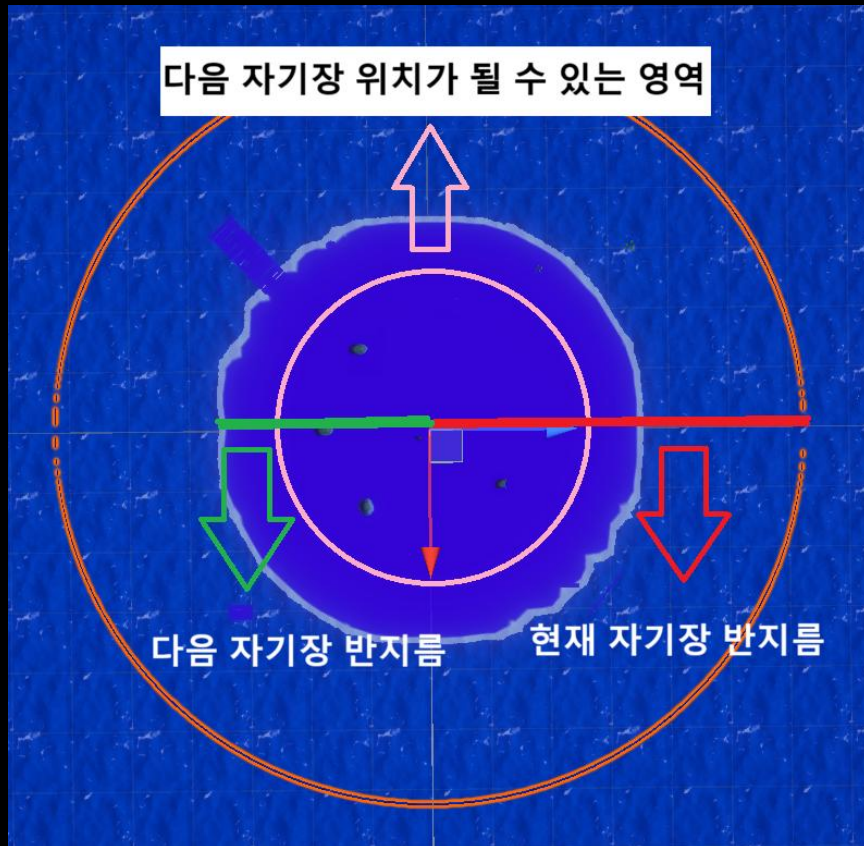
```
public bool PageStart()  
{  
    if (pageSize.Count <= CurPage + 1)  
        return false;  
  
    if (pageSize.Count - 1 == CurPage + 1)  
        gameObject.GetComponent<PhotonViewEx>().  
            Rpc2DSound("BGM/LastPage", (int)Define.Sound2D.Bgm, 1f);  
  
    StopCoroutine(PageCoroutine());  
    NextDestination();  
    StartCoroutine(PageCoroutine());  
    Managers.Game.gameScene.PV.RPC("RegisterMessage", RpcTarget.AllViaServer,  
        $"<color=red>주의!</color> Wn자기장이 줄어듭니다.({curPage} 페이지)", 5f);  
    return true;  
}
```

```
void NextDestination()  
{  
    int nextPage = CurPage + 1;  
  
    if (pageSize[nextPage] == 0f)  
        nextPagePos = transform.position;  
    else  
    {  
        float curRadius = pageSize[CurPage] * 0.5f; // 현재 페이지 반지름 추출  
        float nextRadius = pageSize[nextPage] * 0.5f; // 다음 페이지 반지름 추출  
  
        float gap = curRadius - nextRadius;  
  
        Vector3 nextPos = new Vector3(  
            transform.position.x + Random.Range(-gap, gap),  
            0,  
            transform.position.z + Random.Range(-gap, gap)  
        );  
  
        nextPagePos = nextPos;  
    }  
  
    nextPageSize = new Vector3(pageSize[nextPage], transform.localScale.y, pageSize[nextPage]);  
    CurPage++;  
}
```

소스 코드 : [MagneticField.cs](#)

- 게임이 지루해지지 않게 유저들을 한곳에 모으기 위해서 특정 시간 마다 줄어드는 자기장 기능 구현
- 매번 자기장이 줄어들 때는 현재 자기장의 반지름과 다음 자기장의 반지름의 차이값을 구한다음
- -차이값 ~ +차이값의 범위로 랜덤하게 수치를 뽑아 내 다음 자기장의 목적지로 설정합니다.
- 이렇게 구현한 이유는 다음 자기장이 현재 자기장을 범위를 벗어나지 않는 위치에 선정되도록 구현
- 자기장을 벗어나면 일정 피해를 받게 설정

자기장 위치 선정



- 현재 자기장 반지름 - 다음 자기장 반지름
= 다음 자기장 위치 오프셋
- Ex) $100 - 75 = 25$, 현재 자기장 위치 에서 x, z 가 최소 -25, 최대 25
만큼 이동할 수 있다는 것을 의미하며 동시에 현재 자기장 영역을
벗어나지 않는다.

레드존 구현

- 유저의 움직임을 강제하기 위한 기능
- 특정 주기마다 랜덤 위치에 레드존을 스폰 또는 위치를 조정하여 공격하도록 설계
- 레드존이 폭발하기전에 조명으로 레드존 위치가 표시된다.

```
if (curSpawnCT >= maxSpawnCT)
{
    curSpawnCT = 0f;

    Vector3 rayPos = new Vector3(
        Random.Range(Managers.Game.navMeshMinSize.x, Managers.Game.navMeshMaxSize.x),
        Managers.Game.defaultPointHeight,
        Random.Range(Managers.Game.navMeshMinSize.z, Managers.Game.navMeshMaxSize.z));

    if (Physics.Raycast(rayPos, Vector3.down, out RaycastHit hit, Managers.Game.defaultRayLength, mask))
    {
        if (redZonePV == null)
        {
            GameObject go = Util.FindChild(Managers.Game.photonRoomGroup.gameObject, "RedZone");
            if (go != null)
            {
                go.TryGetComponent(out redZonePV);
            }

            if (redZonePV == null)
            {
                redZonePV = Managers.Resource.PhotonInstantiate(
                    "RedZone", hit.point, Quaternion.identity, redZonePrefab.transform.localScale,
                    Define.PhotonObjectType.RoomObject, Managers.Game.photonRoomGroup.name).GetComponent<PhotonViewEx>();
            }
            else
            {
                redZonePV.RPC("RpcPos", RpcTarget.AllViaServer, hit.point);
            }

            redZonePV.RPC("ActiveBoom", RpcTarget.AllViaServer);
        }
    }
}
```

소스 코드 : [RedZoneCreator.cs](#) , [RedZone.cs](#)

AI 구현

```
protected override void UpdateState()
{
    int index = Random.Range(0, 100);

    if(index <= 50) // 아이들 50가지 경우의 수
        UpdateIdle();
    else if(index <= 90) // 워크 40가지 경우의 수
        UpdateMoving();
    else // 런 10가지 경우의 수
        UpdateRunning();
}
```

소스 코드 : [LPBotController.cs](#)

- 플레이어만 있으면 게임의 재미가 없어지기 때문에 특정 행동을 하는 AI 기능을 구현한다
- FSM으로 idle-move-run 으로 단순하게 구현되어 있다.
- AI 개수는 매칭 인원에 따라 유연하게 배치된다.

플레이어 구현

```
protected override void UpdateState()
{
    Input();
    switch (State)
    {
        case Define.State.Idle:
            UpdateIdle();
            break;
        case Define.State.Moving:
            UpdateMoving();
            break;
        case Define.State.Run:
            UpdateRunning();
            break;
        case Define.State.NormalAttack:
            UpdateAttack();
            break;
        case Define.State.Jumping:
            UpdateJumping();
            break;
    }
}
```

- 플레이어의 조작하기위해
- 행동은 FSM으로 구현 총 3가지 이동, 공격, 달리기 기능이 구현
- 달리기에는 스테미너를 두어 지속적으로 달려 도망가지 못하게 설정, 공격시 피격 판정은 마스터 클라이언트에서 판정
- 피격 대상이 AI면 자신의 체력 -25차감하는 리스크 부여, 플레이어면 피격 플레이어는 즉시 사망 후 추가 점수 부여

15. 좀비 서바이벌 (미니 게임)



- 소개
 - 좀비를 피해 생존
- 특징
 - 좌클릭으로 좀비나 플레이어를 밀어낼 수 있습니다.
- 승리 조건
 - 최후의 플레이어만 생존 혹은 제한 시간안에 버티면 승리합니다.

```
[CreateAssetMenu(menuName = "AdditionPhysicsEffect/Effect")]
Unity 스크립트 | 참조 1개
public class PhysicsEffect : ScriptableObject
{
    public Define.PhysicsType type;
    public Define.PhysicsDir dir;
    public float power;
    public float duraiton;
}
```

넉백기능 구현

```
public override void ApplyOutPhysics(Vector3 pow, Define.PhysicsType type)
{
    switch (type)
    {
        case Define.PhysicsType.Position:
            switch (moveMethod)
            {
                case Define.MoveMethod.CController:
                    crb3D.AddForce(pow, ForceMode.Impulse);
                    break;
                case Define.MoveMethod.RColliderController:
                    rb.AddForce(pow, ForceMode.Impulse);
                    break;
            }
            break;
        case Define.PhysicsType.Rotation:
            switch (moveMethod)
            {
                case Define.MoveMethod.CController:
                    crb3D.AddTorque(pow, ForceMode.Impulse);
                    break;
                case Define.MoveMethod.RColliderController:
                    rb.AddTorque(pow, ForceMode.Impulse);
                    break;
            }
            break;
    }
}
```

- 좀비 서바이벌은 특정 시간동안 좀비를 피해 최후의 생존자가 되는 게임 입니다.
- 단순히 피하는 것보다 밀치는 기능을 구현하여 좀더 극적인 상황을 연출하고자 구현했습니다.
- 물리 기능을 정의하기 위해 PhysicsEffect 클래스 구현
- 캐릭터 컨트롤러와 리지드바디를 활용하여 구현
- Controller3D 클래스에서 물리타입에 따른 해당 오브젝트에 물리 효과 적용
- 효과 적용은 해당 오브젝트 소유자가 직접 적용

소스 코드 : [Controller3D.cs](#)

넉백 기능 적용 순서

- Weapon 클래스
 1. OnAddtionPhysicised 함수를 호출
 2. 무기에 물리 기능 존재시 효과 대상에게 SetPhysics 함수를 실행 시키도록 RPC 호출
- BaseController 클래스
 3. RPC를 받은 소유자의 BaseController의 SetPhysics 가상 함수 호출
 4. 매개변수에 따른 이동(PhysicsStateP) 혹은 회전(PhysicsStateR) 함수 호출
 5. 현재 효과가 적용되지 않으면 코루틴(PhysicsP, PhyscisR) 추상 함수 호출
 6. 일정시간후 최종적으로 ApplyOutPhysics 추상 함수 호출하여 효과 적용

16. 플레이 영상 및 깃 허브 링크



- GitHub : <https://github.com/k660323/FunnyLand>
- YouTube : <https://www.youtube.com/watch?v=By2hg5X7PqI&feature=youtu.be>